

Chapter 21:

Quantum Oracles and Construction of Quantum Gate Matrices

Kamonluk Suksen Ph.D.

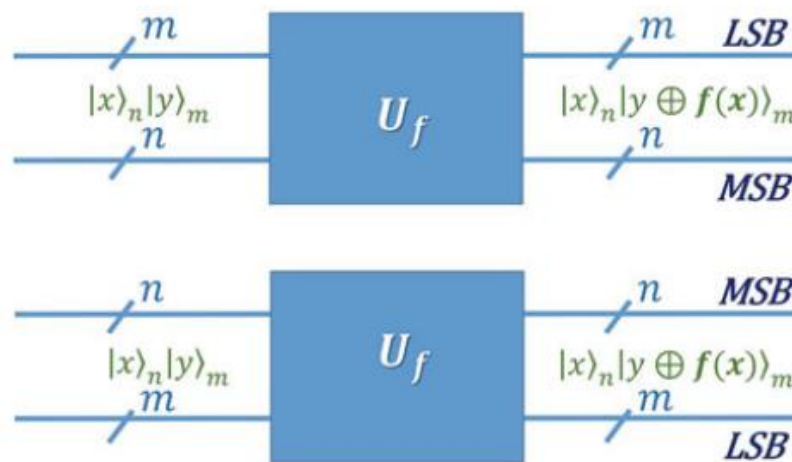
Quantum Oracle

- Since it is an oracle, very often we do not know how it is implemented. We only know that its behavior has some correlations to the problem that we try to solve.
- The problem we are solving usually is related to a function $f(x)$ (such as the $f(x)$, which we want to know if it is constant or balanced in the Deutsch algorithm).
- It can be as simple as a p -qubit identity gate ($I^{\otimes p}$). But usually, it is complex, and we can use some physical systems to implement it.
- We will create an equivalent oracle when we need to test the circuit. But most of the time, we can just treat it as a black box.
- There are two types of quantum oracles. One is called the **XOR quantum oracle** while the other is called the **phase quantum oracle**.
- A quantum oracle is just a quantum gate. Therefore, it must be **unitary** and **reversible**. So, it is represented as U_f , which is a unitary operator corresponding to the function $f(x)$.

Quantum Oracle

• XOR Quantum Oracle

- For any function $f(x)$, it maps n bits of inputs to m bits of outputs. Mathematically, it is written as $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$.
- For example, in the Deutsch problem, we have $n = m = 1$. That is, both input and output have 1-qubit.
- In general, $n \neq m$, we can build a classical gate to map the n -bit inputs to the m -bit outputs. But we cannot do this in quantum computing because every quantum gate needs to be reversible. And this is only possible if $n = m$.
- For example, for $n > m$, many different inputs can produce the same output. Then for a given output, we cannot find the corresponding input in the reverse direction. Or an n -qubit input m -qubit output matrix is not square (it is $m \times n$) and thus it cannot be unitary.
- This issue can be solved by implementing an *XOR quantum oracle*. It has $(n + m)$ -qubit input and output as in Pic. 26.



Pic. 26 A general XOR quantum oracle. The top one is the convention we have been using (IBM-Q) with MSB at the bottom and the bottom one is the other version often seen in other books. Both are using the little-endian convention. Note that $|x\rangle_n |y\rangle_m$ are basis vectors

Quantum Oracle

• XOR Quantum Oracle

- Like for other quantum gates, we define the XOR quantum oracle by describing how it transforms a basis vector to other basis vectors.
- The XOR quantum oracle is $(n + m) \times (n + m)$. If $f(x)$ has n -bit inputs and m -bit outputs, we denote the input basis vector as $|x\rangle_n |y\rangle_m$ (i.e. the n MSB are denoted as $|x\rangle$ and the m LSB are denoted as $|y\rangle_m$). Here $|x\rangle_n$ is the query, which represents the input to $f(x)$ and $|y\rangle_m$ are the **auxiliary qubits**. It is natural to add the m auxiliary bits because we need to store the result $f(x)$ somehow.
- An XOR quantum oracle is defined as that it will keep the n MSB unchanged but the m LSB will be changed to $y \oplus f(x)$. So, the output becomes $|x\rangle_n |y \oplus f(x)\rangle_m$. Note that $y \oplus f(x)$ is a classical bit-wise XOR operator. We can represent its behavior as

$$U_f |x\rangle_n |y\rangle_m = |x\rangle_n |y \oplus f(x)\rangle_m$$

where $|x\rangle_n |y\rangle_m$ is a basis vector in the $\mathbb{C}^{2^{(n+m)}}$ space.

- For example, $n = 3$, $m = 2$, and let $x = 3$ and $f(3) = 3$, find the corresponding input and output basis vectors of the XOR quantum oracle corresponding to $f(x)$.
- We assume the m LSB part of the input is $|0\rangle_m$. So, the input basis vector is $|x\rangle_n |y\rangle_m = |3\rangle_3 |0\rangle_2$ where we substituted the decimal values of x , n , and m . This can be written as $|011\rangle |00\rangle = |01100\rangle$, which is a 5-qubit input.
- The output vector is also a basis vector and can be found as

$$\begin{aligned} U_f |3\rangle_3 |0\rangle_2 &= |3\rangle_3 |0 \oplus f(3)\rangle_2 = |3\rangle_3 |0 \oplus 3\rangle_2 \\ &= |011\rangle |(00 \oplus (11))\rangle \\ &= |011\rangle |11\rangle \\ &= |01111\rangle \end{aligned}$$

Quantum Oracle

- **XOR Quantum Oracle**

- Now, let us prove that the XOR quantum oracle is indeed reversible. We first apply U_f twice to the input.

$$\begin{aligned}
 U_f U_f |x\rangle_n |y\rangle_m &= U_f |x\rangle_n |y \oplus f(x)\rangle_m \\
 &= |x\rangle_n |y \oplus f(x) \oplus f(x)\rangle_m \\
 &= |x\rangle_n |y \oplus 0\rangle_m \\
 &= |x\rangle_n |y\rangle_m \\
 &= I^{\otimes(n+m)} |x\rangle_n |y\rangle_m
 \end{aligned}$$

Where we use the fact that $f(x) \oplus f(x) = 0$ and $y \oplus 0 = y$ because anything XOR with itself results in 0 and anything XOR with 0 is still itself. Therefore, $U_f U_f = I^{\otimes(n+m)}$ (i.e. an $(n + m)$ -dimensional identity matrix). But we know that $U_f U_f^{-1} = I^{\otimes(n+m)}$, where U_f^{-1} is the inverse of U_f . Therefore, $U_f = U_f^{-1}$. This also means that the inverse of U_f exists. So, U_f is reversible.

Quantum Oracle

• Phase Quantum Oracle

- In some applications, we encode $f(x)$ so that it changes the phase of the input basis vector based on $f(x)$ and uses it as an output.
- In this case, we only need n qubits for both the input and the output. The phase quantum oracle matrix has a dimension of $n \times n$. The phase quantum oracle is just a quantum gate and it is defined by how it transforms the basis vectors.

$$U_f |x\rangle_n = (-1)^{f(x)} |x\rangle_n$$

where $|x\rangle_n$ is an n -qubit basis vector.

- In the phase quantum oracle, we see that if $f(x)$ is even, the output is the same as the input. And if $f(x)$ is odd, the output is the same as the input with a phase shift of π . The phase shift is π because $e^{i\pi} = \cos \pi + i \sin \pi = -1$.
- At this point, a quantum oracle can be used to “express” the effect of $f(x)$ (i.e. the answer to our query) in any form as long as it can help us achieve the final goal.
- In a phase quantum oracle, $f(x)$ only affects the phase of the output (therefore, less information is retained).
- Similar to the XOR quantum oracle, a phase quantum oracle needs to be reversible and unitary. To show that it is reversible, we again apply it twice to the input.

$$\begin{aligned} U_f U_f |x\rangle_n &= U_f (-1)^{f(x)} |x\rangle_n \\ &= (-1)^{f(x)} U_f |x\rangle_n \\ &= (-1)^{f(x)} (-1)^{f(x)} |x\rangle_n \\ &= (-1)^{2f(x)} |x\rangle_n = |x\rangle_n \\ &= I^{\otimes n} |x\rangle_n \end{aligned}$$

Quantum Oracle

• Phase Quantum Oracle

- Therefore, it is reversible because its inverse exists (and equals to the oracle itself) by using the reasoning we used in the XOR quantum oracle case. It is also unitary.
- Pic. 27 shows the phase quantum oracle.



Pic. 27 A general phase quantum oracle. Note that $|x\rangle_n$ is a basis vector

- We will show that the set of the basis vectors is still orthonormal after being transformed by the phase quantum oracle and thus the phase quantum oracle is unitary.
- For any two basis vectors, $|a\rangle$ and $|b\rangle$, their inner product is $\langle a|b\rangle$. If they are transformed by a phase quantum oracle, they become $(-1)^{f(a)}|a\rangle$ and $(-1)^{f(b)}|b\rangle$, respectively. The inner product of the transformed basis vector is $(-1)^{(f(a))^*+f(b)}\langle a|b\rangle$. But note, $f(a)$ and $f(b)$ are real numbers, because they are the outputs of the mapping $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$ and are just real binary numbers.
- If $a = b$, then $(-1)^{(f(a))^*+f(b)} = (-1)^{f(a)+f(b)} = (-1)^{2f(a)} = 1$ and the inner product is still the same (i.e. $\langle a|a\rangle = 1$).
- If $a \neq b$, then the inner product is still 0 as $\langle a|b\rangle = 0$. So, the phase quantum oracle keeps the inner products of the basis vectors unchanged (so still orthonormal) and, thus, it is unitary.

Construction of Quantum Gates and Oracles

- How to systematically construct the matrix for a quantum gate?
- As each quantum gate is defined by how it transforms the basis vectors. Since every basis vector has only 1 non-zero entry in its column form, how it is transformed defines the corresponding row of the matrix.
- For example, the 2nd basis vector (we count from 0th) in a 4-dimensional space has the form $\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$.

When it is transformed by a matrix of a quantum gate, it only affects the output through the 2nd row of the matrix. So, we can guess the equation for a quantum gate construction. For a quantum gate, U , with a dimension of $N \times N$,

$$U = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \langle j|U|i\rangle |j\rangle\langle i|$$

where $|i\rangle$ and $|j\rangle$ are the basis vectors. $|j\rangle\langle i|$ is the outer product to define the location of the element in the matrix (j – th row i – th column). $\langle j|U|i\rangle$ tells us how much of $|i\rangle$ will contribute to the formation of $|j\rangle$ after the transformation.

Construction of Quantum Gates and Oracles

- For example, construct the matrix for a 1-bit NOT-gate with only 2 basis vectors.

$$\begin{aligned}
 U_{NOT} &= \sum_{i=0}^{2-1} \sum_{j=0}^{2-1} \langle j | U_{NOT} | i \rangle |j\rangle \langle i| \\
 &= \langle 0 | U_{NOT} | 0 \rangle |0\rangle \langle 0| + \langle 1 | U_{NOT} | 0 \rangle |1\rangle \langle 0| + \\
 &\quad \langle 0 | U_{NOT} | 1 \rangle |0\rangle \langle 1| + \langle 1 | U_{NOT} | 1 \rangle |1\rangle \langle 1| \\
 &= \langle 0 || 1 \rangle |0\rangle \langle 0| + \langle 1 || 1 \rangle |1\rangle \langle 0| + \langle 0 || 0 \rangle |0\rangle \langle 1| + \langle 1 || 0 \rangle |1\rangle \langle 1| \\
 &= |1\rangle \langle 0| + |0\rangle \langle 1| \\
 &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} (1 \ 0) + \begin{pmatrix} 1 \\ 0 \end{pmatrix} (0 \ 1) \\
 &= \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \\
 &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}
 \end{aligned}$$

Construction of Quantum Gates and Oracles

- For example, construct the matrix for a 1-bit Hadamard gate with only 2 basis vectors.

$$\begin{aligned}
 \mathbf{H} &= \sum_{i=0}^{2-1} \sum_{j=0}^{2-1} \langle j | \mathbf{H} | i \rangle | j \rangle \langle i | \\
 &= \langle 0 | \mathbf{H} | 0 \rangle | 0 \rangle \langle 0 | + \langle 1 | \mathbf{H} | 0 \rangle | 1 \rangle \langle 0 | + \\
 &\quad \langle 0 | \mathbf{H} | 1 \rangle | 0 \rangle \langle 1 | + \langle 1 | \mathbf{H} | 1 \rangle | 1 \rangle \langle 1 | \\
 &= \langle 0 | \left(\frac{1}{\sqrt{2}} (| 0 \rangle + | 1 \rangle) \right) | 0 \rangle \langle 0 | + \langle 1 | \left(\frac{1}{\sqrt{2}} (| 0 \rangle + | 1 \rangle) \right) | 1 \rangle \langle 0 | + \\
 &\quad \langle 0 | \left(\frac{1}{\sqrt{2}} (| 0 \rangle - | 1 \rangle) \right) | 0 \rangle \langle 1 | + \langle 1 | \left(\frac{1}{\sqrt{2}} (| 0 \rangle - | 1 \rangle) \right) | 1 \rangle \langle 1 | \\
 &= \frac{1}{\sqrt{2}} (| 0 \rangle \langle 0 | + | 1 \rangle \langle 0 | + | 0 \rangle \langle 1 | - | 1 \rangle \langle 1 |) \\
 &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} \\
 &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}
 \end{aligned}$$

Construction of Quantum Gates and Oracles

- Find the matrix of a XOR quantum oracle in the Deutsch algorithm corresponding to f_B . Then construct the oracle using fundamental quantum gate(s) and simulate on IBM-Q.

$$f_B(0) = 0 \text{ and } f_B(1) = 1$$

- Therefore, the quantum oracle transforms the basis as

$$U_f|0\rangle|0\rangle = |0\rangle|0 \oplus f(0)\rangle = |0\rangle|0 \oplus 0\rangle = |0\rangle|0\rangle$$

$$U_f|0\rangle|1\rangle = |0\rangle|1 \oplus f(0)\rangle = |0\rangle|1 \oplus 0\rangle = |0\rangle|1\rangle$$

$$U_f|1\rangle|0\rangle = |1\rangle|0 \oplus f(1)\rangle = |1\rangle|0 \oplus 1\rangle = |1\rangle|1\rangle$$

$$U_f|1\rangle|1\rangle = |1\rangle|1 \oplus f(1)\rangle = |1\rangle|1 \oplus 1\rangle = |1\rangle|0\rangle$$

where we substituted $f(0) = 0$ and $f(1) = 1$. This is the definition of this oracle.

- In this case, $N = 4$ with 4 basis vectors ($|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$.) and there are $4 \times 4 = 16$ elements in the matrix, only these 4 terms are non-zero.

$$\langle 0|\langle 0|U_f|0\rangle|0\rangle = \langle 0|\langle 0||0\rangle|0\rangle = \langle 00||00\rangle = 1$$

$$\langle 0|\langle 1|U_f|0\rangle|1\rangle = \langle 0|\langle 1||0\rangle|1\rangle = \langle 01||01\rangle = 1$$

$$\langle 1|\langle 1|U_f|1\rangle|0\rangle = \langle 1|\langle 1||1\rangle|1\rangle = \langle 11||11\rangle = 1$$

$$\langle 1|\langle 0|U_f|1\rangle|1\rangle = \langle 1|\langle 0||1\rangle|0\rangle = \langle 10||10\rangle = 1$$

Construction of Quantum Gates and Oracles

- Therefore, the oracle is :

$$\begin{aligned} U_f &= \langle 0| \langle 0| (U_f |0\rangle |0\rangle) |00\rangle \langle 00| + \langle 0| \langle 1| (U_f |0\rangle |1\rangle) |01\rangle \langle 01| + \langle 1| \langle 1| (U_f |1\rangle |0\rangle) |11\rangle \langle 10| + \langle 1| \langle 0| (U_f |1\rangle |1\rangle) |10\rangle \langle 11| \\ &= |00\rangle \langle 00| + |01\rangle \langle 01| + |11\rangle \langle 10| + |10\rangle \langle 11| \end{aligned}$$

$$= \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} (1 \ 0 \ 0 \ 0) + \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} (0 \ 1 \ 0 \ 0) + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} (0 \ 0 \ 1 \ 0) + \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} (0 \ 0 \ 0 \ 1)$$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- This is just a CNOT gate. So, we can implement the Deutsch algorithm for this type of function by using the CNOT gate as the oracle.

Chapter 22:

Grover's Algorithm: I

Kamonluk Suksen Ph.D.

Grover's Algorithm

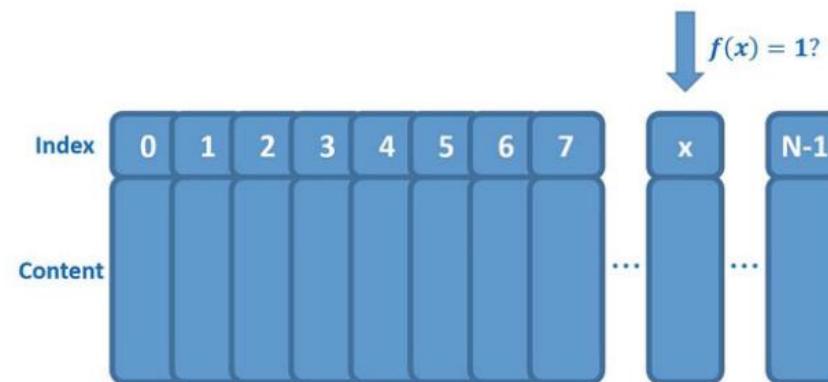
- **Computational Complexity**

- Grover's algorithm is a quantum search algorithm. It provides a quadratic speedup over the classical one.
- A computation problem usually takes more time as its size increases.
- For example, assume we do addition by adding each pair of single digits in a step. To do a 1-digit addition (e.g. $3 + 4$), it might take you 1 second. For a 2-digit addition (e.g. $13 + 24$), it will take you about 2 seconds.
- If your computation efficiency does not decrease as the number of digits increases, it takes you about N seconds to compute an N -bit addition. We say the computational complexity of this problem is $\mathcal{O}(N)$. This means that the computation time goes up linearly with the size of the problem (N).
- If we find a method in which the computational complexity becomes $\mathcal{O}(\log N)$, then the new algorithm achieves an exponential speedup over the original one.
- The Deutsch–Jozsa Algorithm provides an exponential speedup over the classical algorithm. And its computational complexity is just $\mathcal{O}(1)$, which means it is independent of the size of the problem!
- However, not all quantum algorithms can provide an exponential speedup.
- Grover's algorithm is such an algorithm that provides a quadratic speedup over the classical one.
- In reality, just comparing the algorithm speedup is incomplete. We also need to compare the amount of hardware required.

Grover's Algorithm

• The Problem

- Assume we have an unstructured database of N entries.
- We have no additional knowledge on how the data are stored besides we know that the data we are looking for is somewhere in the database.
- Since it is unstructured, we need to search from the beginning to the end. If we are lucky, the first entry might be the one we are looking for. Then it will take us only 1 unit of time. If we are unlucky, the target entry might be at the end of the database and it will take us N units of time. On average, it will take us $\frac{N}{2}$ units of time to find a target entry.
- Therefore, the complexity in a classical search is $\mathcal{O}(\frac{N}{2}) \sim \mathcal{O}(N)$. Note that in complexity calculation, the constant coefficients are usually not important.
- Pic. 28 illustrates the searching process. Any search is related to the computation of a function $f(x)$, where x is the index of an entry in the database. The content of each entry is a function of the index. Therefore, $f(x)$ completely describes the complex computation process. Our goal is to search for the entry that matches certain criteria. The criteria as $f(x) = 1$ if x is the index of the entry we are searching for.



Then, $f(a) = 1$. And $f(x) = 0$ if $x \neq a$.

Pic. 28 Illustration of a searching process in a database with N entries. A function $f(x)$ is used to determine the target is located at entry x .

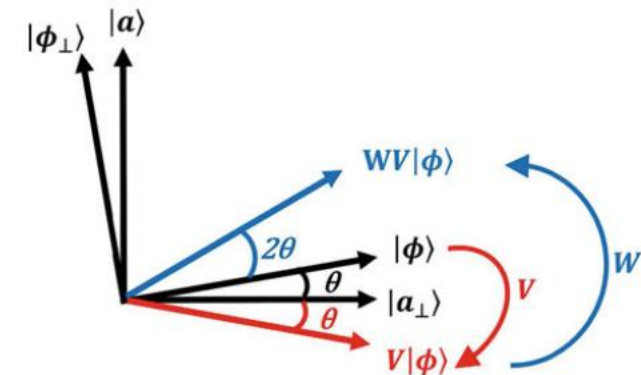
Grover's Algorithm

• An Overview of Grover's Algorithm

- We will first discuss the general idea of Grover's algorithm. Assume what we are searching resides in entry with index $x = a$. For simplicity, we assume $N = 2^n$, and x is between 0 and $2^n - 1$.
- Firstly, we need to encode x for quantum computation. We will encode it as the basis vectors. This is called **basis encoding**.
- Therefore, we only need n -qubit for the encoding. In this process, the index of the target entry, i.e. a , is encoded as the basis vector $|a\rangle$. **$|a\rangle$ is the first important vector in Grover's algorithm.**
- E.g. If there are 13 entries to search and the target is at entry 9, how should we encode the problem for quantum computing? Since there are 13 entries, we can use 4 qubits that have basis vectors from $|0000\rangle$ to $|1111\rangle$. The target is encoded in the basis vector $|1001\rangle$.
- **The second important vector in Grover's algorithm is the $|a_\perp\rangle$.** This is a vector formed by the equal linear superposition of all basis vectors except $|a\rangle$. So, it is orthogonal (perpendicular) to $|a\rangle$ and it has a zero inner product with $|a\rangle$. It is expressed as:

$$|a_\perp\rangle = \frac{1}{\sqrt{2^n - 1}} \sum_{x=0(x \neq a)}^{2^n - 1} |x\rangle$$

- Note that the coefficient is $\frac{1}{\sqrt{2^n - 1}}$ (instead of $\frac{1}{\sqrt{2^n}}$) because there are only $2^n - 1$ terms after excluding $|a\rangle$. Pic. 29 illustrates the Grover's algorithm. Note that $|a\rangle$ is drawn perpendicular to $|a_\perp\rangle$ in the 2D plane.



Pic. 29 Illustration of the Grover's algorithm. The first two steps are shown. Applying V to $|\phi\rangle$ rotates $|\phi\rangle$ to its mirror image about $|a_\perp\rangle$. Applying W to the new vector rotates it to its mirror image about $|\phi\rangle$. This effectively rotates $|\phi\rangle$ by 2θ closer to the solution $|a\rangle$. This is repeated until it aligns with the solution $|a\rangle$.

Grover's Algorithm

• An Overview of Grover's Algorithm

- Each x is the decimal form of the basis vector from 0 to $2^n - 1$ and the corresponding basis vectors in binary form are from $|00 \cdots 00\rangle$ to $|11 \cdots 11\rangle$ with n qubits.
- Note that $|a_\perp\rangle$, $|a\rangle$, and $|\phi\rangle$ are drawn *on the same plane* as illustrated in Pic. 29 because $|\phi\rangle$ is a linear combination of $|a_\perp\rangle$ and $|a\rangle$.
- The idea in the Grover's algorithm is to reflect $|\phi\rangle$ about $|a_\perp\rangle$. If the initial angle between $|\phi\rangle$ and $|a_\perp\rangle$ is θ , this is equivalent to rotating $|\phi\rangle$ clockwise by 2θ . We assume this can be accomplished by a matrix V . Then we further reflect the new vector ($V|\phi\rangle$) about $|\phi\rangle$. We assume we can find a matrix W to perform this operation. The final vector becomes $WV|\phi\rangle$, which is equivalent to rotating $|\phi\rangle$ by 2θ closer to $|a\rangle$. This process is repeated many times.
- Each time after WV is applied, the vector will be rotated by 2θ closer to $|a\rangle$ and eventually, we hope that it will align with $|a\rangle$. In this case, the wavefunction contains only $|a\rangle$ and we have found the entry.
- Therefore, the idea of Grover's algorithm is to begin with a wavefunction in an equal superposition of all basis vectors and then transform it gradually by suppressing the non-solution. Eventually, we have 100% certainty to get our solution when we perform a measurement as the final wavefunction is composed of only $|a\rangle$.
- It is obvious that the function $f(x)$ needs to be a part of this algorithm. It will be embedded in a quantum oracle. Moreover, the fact (or requirement) that $f(a) = 1$ must be embedded in V because $|a_\perp\rangle$ is the vector about which V reflects the vectors.

Grover's Algorithm

• Implementation of Grover's Algorithm

- We can use the Hadamard gate to create an equal superposition of all basis states.

$$H^{\otimes n}|0\rangle_n = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle = |\phi\rangle$$

- **Properties of θ and Algorithm Complexity:** If the problem we care about has many entries (i.e. N is large), then we expect θ to be small. This is because θ is the angle between $|a_\perp\rangle$ (which is a linear superposition of all basis vectors except $|a\rangle$) and $|\phi\rangle$ (which is a linear superposition of all basis vectors). Their inner product must be close to 1 when the dimension is high as they only have a difference in 1 basis vector among N . We note that the inner product of two *normalized* vectors on a 2D plane is $\cos \theta$.

$$\begin{aligned} \cos \theta &= \langle \phi | |a_\perp \rangle = \left(\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} \langle x | \right) \left(\frac{1}{\sqrt{2^n-1}} \sum_{y=0(y \neq a)}^{2^n-1} |x\rangle \right) \\ &= \frac{1}{\sqrt{2^n}} \frac{1}{\sqrt{2^n-1}} (2^n - 1) \\ &= \frac{\sqrt{2^n-1}}{\sqrt{2^n}} \end{aligned}$$

Grover's Algorithm

• Implementation of Grover's Algorithm

- Then, using the orthonormality of the basis vectors, there are only $2^n - 1$ terms left after expanding the first line.
- Using the identity, $\sin^2 \theta + \cos^2 \theta = 1$, we find that

$$\sin \theta = \frac{1}{\sqrt{2^n}}$$

- When $N = 2^n$ is large, this is a small number. We know that $\sin \theta \approx \theta$ when θ is small. Therefore, for a large N ,

$$\theta \approx \frac{1}{\sqrt{2^n}}$$

- Since θ is small, the angle between $|\phi\rangle$ and $|a\rangle$ is $\frac{\pi}{2} - \theta \approx \frac{\pi}{2}$. Each WV operation rotates $|\phi\rangle$ by 2θ towards $|a\rangle$. Therefore, the number of WV operation required to rotate $|\phi\rangle$ to $|a\rangle$ is

$$\frac{\pi/2}{2\theta} = \frac{\pi\sqrt{2^n}}{4} = \frac{\pi}{4}\sqrt{N}$$

This explains why the complexity of Grover's algorithm is $\mathcal{O}(\sqrt{N})$, where we assume each WV is a significant computation in this problem like the single-digit addition in the addition problem.

Grover's Algorithm

• Implementation of Grover's Algorithm

- **Implementation of V** As shown in Pic. 29, the V operation reflects $|\phi\rangle$ about $|a_\perp\rangle$. V must be a quantum gate or a set of quantum gates and, thus, it is defined by telling how it transforms basis vectors, $|x\rangle$. The following is the given definition:

$$V|x\rangle_n = (-1)^{f(x)}|x\rangle_n$$

It is obvious that V keeps all basis vector unchanged except when $x = a$, where $f(a) = 1$, it adds a phase shift of π to $|a\rangle$ (i.e. the coefficient is multiplied by $-1 = e^{i\pi}$). Then apply V to an arbitrary vector $|\beta\rangle$, which can be $|\phi\rangle$ or the rotated $|\phi\rangle$. $|\beta\rangle$ can be expressed as a linear combination of the basis vectors.

$$|\beta\rangle = \sum_{i=0}^{2^n-1} \beta_i |i\rangle$$

where β_i 's are the complex coefficients and $|i\rangle$'s are the basis vectors. Then

$$\begin{aligned} V|\beta\rangle &= V \sum_{i=0}^{2^n-1} \beta_i |i\rangle \\ &= \sum_{i=0}^{2^n-1} \beta_i V|i\rangle = \sum_{i=0}^{2^n-1} (-1)^{f(i)} \beta_i |i\rangle \\ &= \left(\sum_{i=0, i \neq a}^{2^n-1} \beta_i |i\rangle \right) - \beta_a |a\rangle \end{aligned}$$

Grover's Algorithm

• Implementation of Grover's Algorithm

- **Implementation of V** We will use the quantum oracle to implement V . Therefore, $V = U_f$, where U_f is the quantum phase oracle of the problem and we assume it is given and available. The quantum oracle is assumed to contain the information of $f(x)$ and can compute $f(x)$. Therefore, it knows the value of a that gives $f(a) = 1$. Our goal is to query the quantum oracle so that it reveals the value of a .
- **Implementation of W and the Quantum Oracle** The action of W is to reflect any vector on the plane in Pic. 29 about $|\phi\rangle$. A vector, $|\gamma\rangle$, on that plane can be expressed as a linear combination of the two orthonormal vectors, $|\phi\rangle$ and $|\phi_\perp\rangle$.

$$|\gamma\rangle = \gamma_0|\phi\rangle + \gamma_1|\phi_\perp\rangle$$

where γ_0 and γ_1 are the complex coefficients. We expect that after applying W , it will become $\gamma_0|\phi\rangle - \gamma_1|\phi_\perp\rangle$ because reflection about $|\phi\rangle$ means that the $|\phi\rangle$ coefficient should be unchanged and the $|\phi_\perp\rangle$ coefficient should be negated (gain a phase shift of π). To achieve this, we can set

$$W = 2 |\phi\rangle\langle\phi| - I$$

Grover's Algorithm

- **Implementation of Grover's Algorithm**

- **Implementation of W and the Quantum Oracle** Shows that W reflect $|\gamma\rangle$ about $|\phi\rangle$.

$$\begin{aligned}
 W|\gamma\rangle &= (2|\phi\rangle\langle\phi| - I)|\gamma\rangle \\
 &= 2|\phi\rangle\langle\phi|\gamma\rangle - I|\gamma\rangle \\
 &= 2|\phi\rangle\langle\phi|(\gamma_0|\phi\rangle + \gamma_1|\phi_\perp\rangle) - (\gamma_0|\phi\rangle + \gamma_1|\phi_\perp\rangle) \\
 &= 2|\phi\rangle\gamma_0 - (\gamma_0|\phi\rangle + \gamma_1|\phi_\perp\rangle) \\
 &= \gamma_0|\phi\rangle - \gamma_1|\phi_\perp\rangle
 \end{aligned}$$

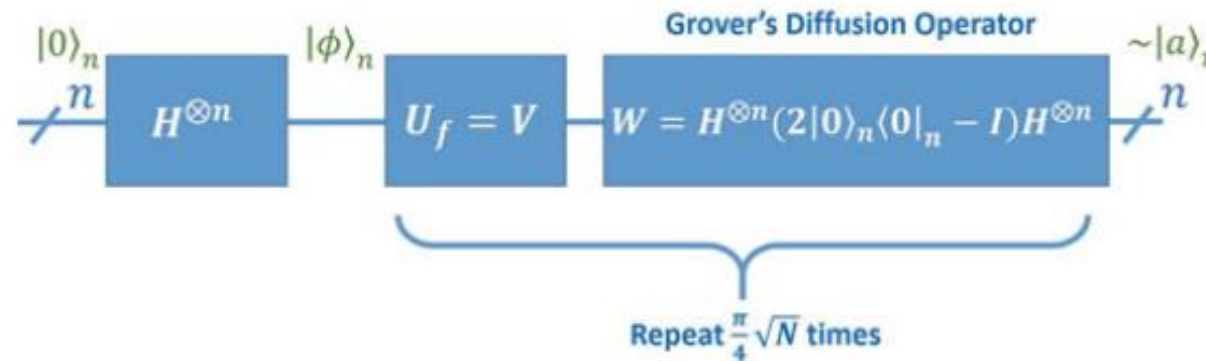
We construct W using more fundamental gates. This is called the **Grover diffusion operator**.

$$W = H^{\otimes n}(2|0\rangle_n\langle 0|_n - I)H^{\otimes n}$$

Grover's Algorithm

- **Implementation of Grover's Algorithm**
 - **Circuit for Grover's Algorithm**

Pic. 30 shows the circuit for Grover's algorithm when a quantum phase oracle U_f is available. Although it looks sophisticated that we need to implement W many times, this is actually not a problem for a quantum circuit. As discussed earlier, unlike in a classical circuit, the horizontal direction in a quantum circuit represents the flow of time instead of the space and a quantum gate is nothing but just a sequence of pulses (e.g. microwave or laser). The repetition just refers to the application of pulses multiple times to the same atoms or electrons. Therefore, even $N = 10^{12}$, we just need to apply the sequence of pulses corresponding to W for 10^6 times instead of having 10^6 gates in the circuit.



Pic. 30 Circuit to implement the Grover's algorithm when a quantum phase oracle, U_f , is available

Chapter 23: Grover's Algorithm: II

Kamonluk Suksen Ph.D.

Numerical Example for Grover's Algorithm

- Assume there is a database with four entries. Therefore, it has indices, 0, 1, 2, and 3. It is unstructured, and we do not know where what we are looking for is stored. Assume it is stored in entry 1, i.e. index $a = 1$. We assume there is a phase quantum oracle, $\mathbf{U}_f$, corresponding to $f(x = 1) = 1$ and $f(x \neq 1) = 0$ available.
- $N = 4$ in this problem. Therefore, $n = 2$ as $2^2 = 4$. So, we only need 2 qubits. As $a = 1$, therefore, $|a\rangle = |01\rangle$. For $|a_\perp\rangle$, which is a superposition of all basis vectors but excluding $|a\rangle = |01\rangle$, it is

$$\begin{aligned}
 |a_\perp\rangle &= \frac{1}{\sqrt{2^n - 1}} \sum_{x=0(x \neq a)}^{2^n - 1} |x\rangle \\
 &= \frac{1}{\sqrt{2^n - 1}} (|00\rangle + |10\rangle + |11\rangle) \\
 &= \frac{1}{\sqrt{3}} (|00\rangle + |10\rangle + |11\rangle)
 \end{aligned}$$

Numerical Example for Grover's Algorithm

- For $|\phi\rangle$, which is a superposition of all basis vectors, it is

$$\begin{aligned} |\phi\rangle &= \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \\ &= \frac{1}{2^{2/2}} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \\ &= \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \end{aligned}$$

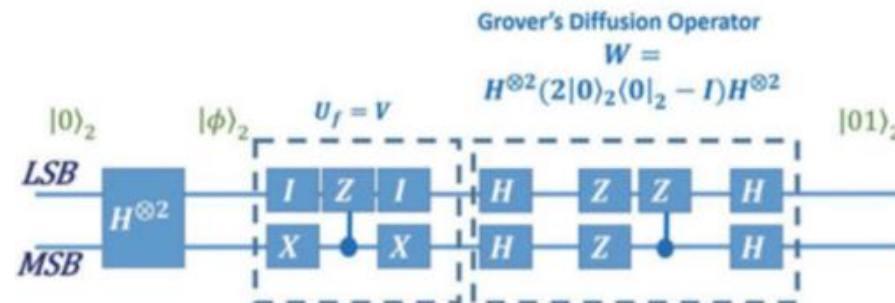
Numerical Example for Grover's Algorithm

• Construction of Quantum Oracle

- In reality, we do not know the structure of the quantum oracle, $\mathbf{U}_f$. It will be realized through some physical systems. We are supposed to have no knowledge of the value of a . But to construct the oracle for calculation and simulation, we need to use the knowledge in this example that $a = 1$. So, we have

$$\mathbf{U}_f = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This is because $|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$, $|01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$, $|10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$, and $|11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$ and particularly, $\mathbf{U}_f|01\rangle = (-1)^{f(1)}|01\rangle = (-1)^1|01\rangle = -|01\rangle$. So, $\mathbf{V} = \mathbf{U}_f$. $\mathbf{U}_f$ looks like a controlled-Z-gate, $\mathbf{CZ}$, except that it gains a negative sign when the MSB is 0 and the LSB is 1 instead of when both are 1.



Pic. 31 Circuit to implement the Grover's algorithm for the given example

Numerical Example for Grover's Algorithm

• Construction of Quantum Oracle

- In this case from Pic. 31, we can perform a NOT operation on the MSB first and then apply a controlled Z-gate. After that, we will apply a NOT gate again to bring the basis vector back to themselves. Therefore,

$$\begin{aligned}
 U_f &= (U_{NOT} \otimes I) U_{CPS,\pi} (U_{NOT} \otimes I) \\
 &= \left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) \\
 &= \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \\
 &= \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = U_f
 \end{aligned}$$

The rightmost NOT gate in the equation (which will be the leftmost NOT gate in the circuit) shuffles the basis vectors so that $|01\rangle$ can get the negative sign from the CZ gate in line 3 (therefore, in line 3, we performed the multiplication of the rightmost 2 matrices first). The last NOT gate then shuffles the basis vector components back to the original position (lines 4).

Numerical Example for Grover's Algorithm

• Construction of the Grover Diffusion Operator

- Previously, we show that the Grover diffusion operator can be expressed as

$$W = H^{\otimes n}(2|0\rangle_n\langle 0|_n - I)H^{\otimes n} = H^{\otimes 2}(2|0\rangle_2\langle 0|_2 - I)H^{\otimes 2}$$

It is easy to implement the leftmost and rightmost Hadamard gates, which are just

$$\begin{aligned} H^{\otimes 2} &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \end{aligned}$$

- Find the matrix for $2|0\rangle_2\langle 0|_2 - I$.

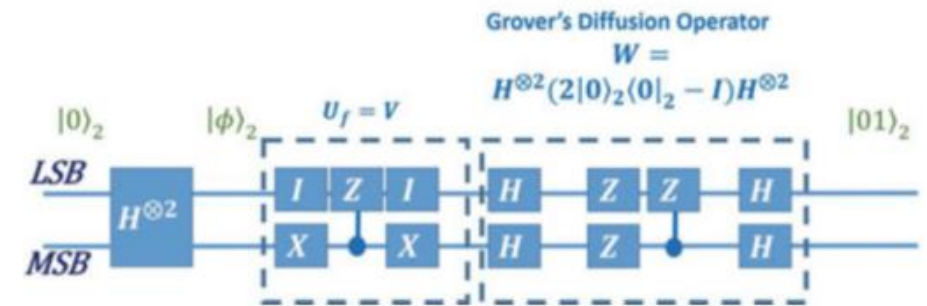
$$\begin{aligned} 2|0\rangle_2\langle 0|_2 - I &= 2 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} (1 \ 0 \ 0 \ 0) - \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \end{aligned}$$

Numerical Example for Grover's Algorithm

• Construction of the Grover Diffusion Operator

- To implement it, we have to use more elementary quantum gates, we need to further decompose it. To get three -1 's and one 1 in the diagonal, one possibility is to use the tensor product of two Z gates (which will give us two -1 's) and then apply $\mathbf{CZ} = \mathbf{U}_{CPS,\pi}$ gate again to get another -1 . Therefore, $2|0\rangle_2\langle 0|_2 - \mathbf{I} = \mathbf{U}_{CPS,\pi}(\mathbf{Z} \otimes \mathbf{Z})$. (Pic. 31)
- Try to proof that $2|0\rangle_2\langle 0|_2 - \mathbf{I} = \mathbf{U}_{CPS,\pi}(\mathbf{Z} \otimes \mathbf{Z})$!

$$\begin{aligned}
 \mathbf{U}_{CPS,\pi}(\mathbf{Z} \otimes \mathbf{Z}) &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \left(\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right) \\
 &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} = 2|0\rangle_2\langle 0|_2 - \mathbf{I}
 \end{aligned}$$



Pic. 31 Circuit to implement the Grover's algorithm for the given example

Numerical Example for Grover's Algorithm

• Evolution of the Wavefunction

- We use matrix multiplications to see how the wavefunction evolves in Pic. 31.
- The wavefunction is first initialized to $|0\rangle_2 = |00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$ at the far left. After passing through the Hadamard gate, it becomes

$$\begin{aligned} H^{\otimes 2} &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \frac{1}{2} |00\rangle + |01\rangle + |10\rangle + |11\rangle \end{aligned}$$

This is just the $|\phi\rangle$. Then it goes through the quantum oracle (i.e. V) and it becomes, which it flips the sign of $|a\rangle = |01\rangle$.

$$\begin{aligned} V|\phi\rangle &= U_f \left(\frac{1}{2} |00\rangle + |01\rangle + |10\rangle + |11\rangle \right) \\ &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \end{aligned}$$

Numerical Example for Grover's Algorithm

• Evolution of the Wavefunction

- We then apply the Grover diffusion operator, therefore, we have

$$\begin{aligned}
 & \mathbf{H}^{\otimes n}(2|0\rangle_2\langle 0|_2 - \mathbf{I})\mathbf{H}^{\otimes 2}(\mathbf{V}|\phi\rangle) \\
 &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \\
 &= \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \\
 &= \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix} \\
 &= \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |01\rangle = |a\rangle
 \end{aligned}$$

Interestingly, we have achieved the solution by applying $\mathbf{WV}$ only one time instead of the approximately 2 times predicted by $\frac{\pi}{4}\sqrt{N}$.

This is because N is small here, and it is no longer valid to ignore θ in the calculation of the number of $\mathbf{WV}$ operations required.

That is, $\pi/2 - \theta \approx \pi/2$ is no longer valid.

Numerical Example for Grover's Algorithm

- **Evolution of the Wavefunction**

- Find θ using the orthonormal property of the basis vectors.

$$\begin{aligned}\cos \theta &= \langle a_{\perp} | \phi \rangle \\ &= \frac{1}{\sqrt{3}} (\langle 00 | + \langle 10 | + \langle 11 |) \frac{1}{2} (\langle 00 | + \langle 01 | + \langle 10 | + \langle 11 |) \\ &= \frac{\sqrt{3}}{2}\end{aligned}$$

Therefore, $\theta = \frac{\pi}{6}$. One **WV** operation will rotate it by $2\theta = \frac{\pi}{3}$ and it will be aligned with $|a\rangle$ because $\frac{\pi}{6} + \frac{\pi}{3} = \frac{\pi}{2}$.

Implementation Using XOR Quantum Oracle

- We have been assuming that the quantum oracle is given as a phase oracle. It is also possible to implement Grover's algorithm using XOR quantum oracle.
- We need to think of how to incorporate the XOR quantum oracle in V .
- Firstly, the XOR quantum oracle is a quantum gate of $n + m$ qubits, where n is the number of qubits we need to encode the index of the database (i.e. $2^n = N$) and m is the number of the auxiliary qubit to encode the output $f(x)$. Since $f(x)$ is either 0 or 1, we only need 1-qubit and thus $m = 1$.
- The definition of an XOR quantum oracle is repeated here

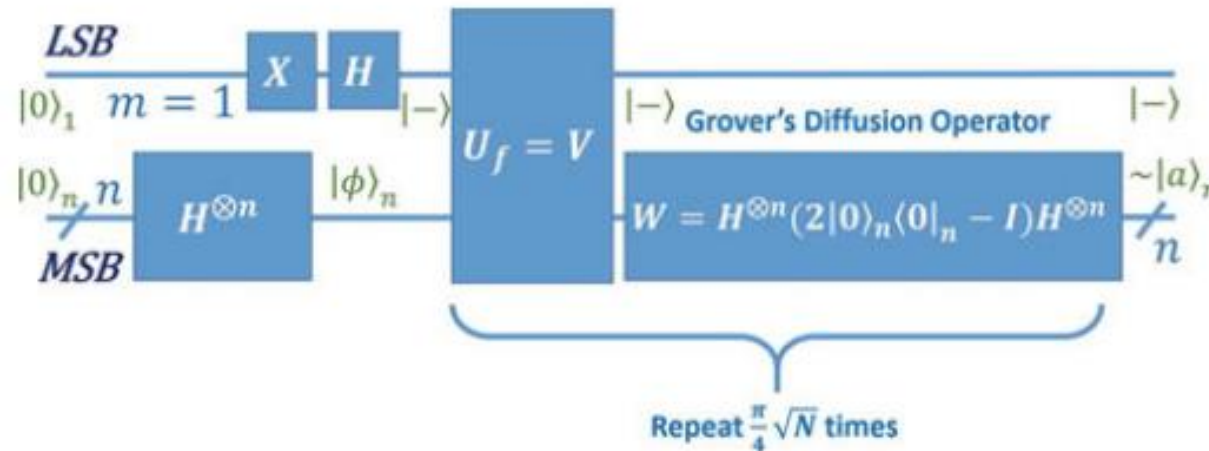
$$U_f |x\rangle_n |y\rangle = |x\rangle_n |y \oplus f(x)\rangle$$

where we have used $m = 1$. Therefore, the oracle has $n + 1$ qubits. In order to use the oracle in V , we need to apply $|-\rangle$, we can apply a U_{NOT} gate followed by a H gate to the initial $|0\rangle$ state of the auxiliary qubit.

Implementation Using XOR Quantum Oracle

- For any basis vector $|x\rangle_n$, it combines (perform tensor product with) the auxiliary qubit as the input to the oracle.

$$\begin{aligned}
 U_f|x\rangle_n|-\rangle &= U_f(|x\rangle_n \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)) \\
 &= \frac{1}{\sqrt{2}}(U_f(|x\rangle_n|0\rangle) - (U_f|x\rangle_n|1\rangle)) \\
 &= \frac{1}{\sqrt{2}}(|x\rangle_n|f(x)\oplus 0\rangle - |x\rangle_n|f(x)\oplus 1\rangle) \\
 &= \frac{1}{\sqrt{2}}(|x\rangle_n|f(x)\rangle - |x\rangle_n|f(x)\oplus 1\rangle)
 \end{aligned}$$



Pic. 32 Circuit to implement the Grover's algorithm with a XOR quantum oracle, U_f .

Implementation Using XOR Quantum Oracle

- If $f(x) = 0$, then $f(x) \oplus 1 = 1$ and $(-1)^{f(x)} = 1$. Therefore,

$$\begin{aligned} U_f(|x\rangle_n |-\rangle) &= \frac{1}{\sqrt{2}} (|x\rangle_n |f(x)\rangle - |x\rangle_n |f(x) \oplus 1\rangle) = \frac{1}{\sqrt{2}} (|x\rangle_n |0\rangle - |x\rangle_n |1\rangle) \\ &= |x\rangle_n \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= |x\rangle_n |-\rangle \\ &= (-1)^{f(x)} |x\rangle_n |-\rangle \end{aligned}$$

- If $f(x) = 1$, then $f(x) \oplus 1 = 0$ and $(-1)^{f(x)} = -1$. Therefore,

$$\begin{aligned} U_f(|x\rangle_n |-\rangle) &= \frac{1}{\sqrt{2}} (|x\rangle_n |f(x)\rangle - |x\rangle_n |f(x) \oplus 1\rangle) = \frac{1}{\sqrt{2}} (|x\rangle_n |1\rangle - |x\rangle_n |0\rangle) \\ &= -|x\rangle_n \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= -|x\rangle_n |-\rangle \\ &= (-1)^{f(x)} |x\rangle_n |-\rangle \end{aligned}$$

- Combining together, regardless of the value of $f(x)$, we have

$$\begin{aligned} U_f(|x\rangle_n |-\rangle) &= (-1)^{f(x)} |x\rangle_n |-\rangle \\ &= (V|x\rangle_n) \otimes (I|-\rangle) = V \otimes I |x\rangle_n |-\rangle \end{aligned}$$

By adding the auxiliary bit with $|-\rangle$ as the input value, the XOR quantum oracle behaves exactly as a phase quantum oracle on the n query qubits. The output of the auxiliary qubit is unchanged (still $|-\rangle$). Therefore, we can repeat the WV as before by about $\sqrt{N}$ times to find the solution.